G00GL3YYY

An In-Memory High-Performance Search Engine Prototype

PROJECT MEMBERS & REGISTRATION NUMBERS

Muhammad Shahzeb	SP22-BSE-073
Syed Hadi Raza	FA22-BSE-057
Wassay	FA22-BCS-140

Course: Data Structures and Algorithms (DSA) **Timeline:** 1 Week Sprint **Tech Stack:** Node.js Architecture

1. Executive Summary

G00gl3yyy is an optimized, single-process web search engine prototype engineered to process, index, and query local document repositories (such as structured Wikipedia article dumps) in sub-millisecond execution times. Instead of relying on traditional relational or document databases, G00gl3yyy implements its entire production pipeline and core data structures natively in-memory. The primary structural objective of this project is to demonstrate how advanced computer science concepts like **Tries**, **Postings Lists**, and **Min-Heaps** minimize spatial and temporal complexities in modern information retrieval architectures.

2. Core Architecture & Pipeline

The application is split into three distinct modules, operating sequentially as a decoupled data pipeline:

- **The Document Crawler:** Traverses a designated local web/text repository, maps structural dependencies, filters raw data, and builds a centralized collection storage.
- **The Lexical Indexer:** Ingests document streams, executes linguistic preprocessing, and builds an Inverted Index mapped inside a custom digital tree interface.
- **The Query Engine & Ranker:** Evaluates multi-word user search terms, computes mathematical relevancy scores, and surfaces top matches via strict bounded sorting mechanisms.

3. Detailed Modules & DSA Breakdown

A. The Web Crawler & Document Store

The crawler acts as the data acquisition layer, systematically exploring local files or text dumps instead of facing live-network rate limits.

- **Graph Representation:** The local document network is treated as a *Directed Graph* where documents represent vertices (V) and hyperlinks map as directional edges (E).
- **Traversal Algorithm:** A strict **Breadth-First Search (BFS)** traversal is used to index pages level-by-level, utilizing an explicit First-In, First-Out ($FIFO$) Array Queue.
- **Cycle & Loop Prevention:** A native Set data structure handles unique URL tracking, providing $O(1)$ average-time lookups to ensure no duplicate file is crawled twice.

B. The Lexical Indexer (The Inverted Index)

The indexer optimizes search efficiency from a manual full-text scan ($O(N)$ per query) to a structured memory dictionary lookup.

- **The Trie (Prefix Tree):** Instead of a generic hash table, the dictionary vocabulary is mapped onto a character-driven **Trie**. This compresses space by sharing common word prefixes and natively enables instantaneous prefix-matching.
- **The Postings List:** Every terminal node in the Trie pointing to a valid word stores a sequential array of integers representing **Document IDs** where that word exists.

C. The Query Engine & Scoring Ranker

When a user inputs a query, the engine avoids global array sorting to keep memory execution lean.

- **Two-Pointer List Intersection:** Multi-word queries fetch multiple postings lists from the Trie. Because IDs are assigned sequentially, these arrays are naturally pre-sorted. A linear **Two-Pointer Algorithm** computes the intersection of matching documents in highly efficient $O(M + N)$ time.
- **Term-Frequency (TF) Ranking:** The engine evaluates document relevance by analyzing keyword occurrences within the matched document boundaries.
- **Top-K Min-Heap Filter:** To return only the highest matching results without sorting the entire dataset ($O(N \log N)$), results are streamed through a custom **Min-Heap bounded to size K**. This cuts the sorting execution complexity down to a highly optimized $O(N \log K)$.

4. Expected Complexity Matrix

Operation	Target Data Structure / Algorithm	Time Complexity	Space Complexity
URL Tracking	Hash Set (*Set*)	$O(1)$	$O(V)$
Word Indexing	Trie Node Insertion	$O(L)$ *(word length)*	$O(\Sigma \times L)$
Multi-word Search	Two-Pointer Intersection	$O(M + N)$	$O(M \cap N)$
Result Ranking	Bounded Min-Heap (Size K)	$O(N \log K)$	$O(K)$

5. Key Deliverables & Interactive Features

Engine Diagnostics & UI Polish:

1. **Sub-Millisecond Diagnostics:** The system UI will display explicit execution timers showing exactly how many microseconds (*µs*) the Trie traversal consumed.
2. **Real-time Autocomplete:** Leverages the Trie structure to suggest search vocabulary seamlessly as the evaluator types.
3. **Zero-Dependency Pipeline:** A completely clean, custom-coded pipeline written in vanilla JavaScript/ Node.js, showcasing custom algorithmic implementations without external packages.

6. Implementation Timeline (1-Week Sprint)

- **Days 1–2:** Local dataset parsing, configuration setup, and building the BFS Local Crawler.
- **Days 3–4:** Core structure assembly—Coding the custom character-based *TrieNode* architecture and the Text Tokenizer pipeline.
- **Days 5–6:** Writing the Query Engine, the linear Two-Pointer list intersection logic, and the Top-K *MinHeap* sorting filter.
- **Day 7:** Connecting a minimalist Web UI Dashboard, performance metric testing, and final code optimization.